Task 1

Inputs are vehicle registration, vehicle category, user role (student, staff or visitor), entry and exit date/time, pass details, a charging request and consumed kWh. Facility configuration supplies each floor capacity. Outputs are a unique ticket, allocated floor/slot, availability or rejection, exit fee and total collection.

The nouns identify these classes: Vehicle stores identity and supplies the hourly-rate behaviour; TwoWheeler and Car supply different rates; ElectricCar adds Chargeable behaviour; Slot owns one position and its occupancy; ParkingTicket links vehicle, slot, entry, role and pass; ParkingFacility controls allocation, duplicate detection and exit; Pass represents parking entitlement; MonthlyPass checks registration, role and month; PricingRule separates the billing policy from allocation. A separate Floor class is unnecessary because a row of Slot[][] already represents a floor. Roles are validated strings because their only original behavioural difference is pass eligibility. Verbs become enter(), exit(), occupy(), release(), calculateFee(), chargingFee() and covers(). Each class protects the information needed for its own responsibility.

A slot holds at most one vehicle. One registration has at most one active ticket within a facility. Registration is normalized to uppercase. New tickets have different identities across facilities; the process UUID plus shared counter prevents reuse in a run and makes cross-run collisions negligibly unlikely rather than mathematically impossible. Strict durable uniqueness would require persistent numbering. Entry must not follow exit; durations and kWh cannot be negative. A charging booking requires a Chargeable vehicle and a charging slot. Invalid exit requests leave the ticket and slot unchanged. Passes waive only parking, never electricity. Occupancy is derived per facility, rather than shared globally.

Assumptions: general slots accommodate all categories; the last slot on every floor with at least two slots is reserved for charging. A one-slot floor is general. Floors and slots use zero-based indexes. Rates are Rs 10/hour for two-wheelers and Rs 30/hour for cars/EVs; every started hour is billed and zero minutes costs zero. No daily cap is imposed because no cap value was specified. Valid monthly passes are registration-bound, restricted to staff/students and must cover both endpoints of the stay. All times are local campus times. The application is single-threaded and in-memory.

Alternative: parallel arrays String[] regNo, String[] type and LocalDateTime[] entryTime store matching records at the same index. Search uses FOR i FROM 0 TO n-1; IF regNo[i] = key THEN RETURN i; ENDIF; ENDFOR. Searching an object array and the parallel arrays both take O(n L) worst-case, where L is registration length; for bounded registrations this is O(n). Both need O(1) search workspace. Each vehicle uses O(1) fields/references plus O(L) registration storage in either representation; total O(n) for bounded strings. Objects add header/alignment overhead, while parallel arrays add three array headers and require index synchronization. Adding a colour field requires one field and constructor change in Vehicle; parallel arrays require a fourth array, coordinated resizing and coordinated updates at every insertion/deletion site. Adding the field for n existing records takes O(n) total initialization in both designs; per-vehicle storage remains O(1).

Task 2

Slot and floor numbers use int because they are array indexes. Registration uses String to preserve letters and leading zeroes. Entry and exit use LocalDateTime to retain dates across midnight. Duration uses Duration and long minutes. The integrated model uses long whole rupees because every configured rate is integral; multiplication/addition use exact arithmetic where totals could overflow. A currency with fractional rupees would need long paise or BigDecimal. Task08 retains double for the supplied fee calculation.

Slot[][] is a jagged array: slots[f] = new Slot[capacities[f]]. For {20,50,100}, rows have those lengths, so no position exists beyond its floor capacity. Each Slot contains floor, number, charging flag and a private Vehicle reference. Direct access with validated floor/slot indexes takes O(1); scanning for a suitable free position takes O(N), N = sum of capacities.

The application reads optional command-line arguments: java -cp build parking.app.Main CAR TN01A1. Demonstrations use fixed timestamps for repeatable tests. This avoids Scanner token/newline issues. Arguments are validated by the factory and constructors. kWh uses int, with an explicit widening multiplication 12L * kWh. Duration is rounded upward to a minute before integer hours are rounded upward. For 61 minutes, 61 / 60 is 1 and wrongly bills Rs 30; (int)(61 / 60.0) also truncates to 1. hours = minutes / 60 + (minutes % 60 == 0 ? 0 : 1) yields 2 and Rs 60. It also avoids overflow from minutes + 59. A one-nanosecond stay is positive and bills one hour. Date-aware overloads avoid LocalTime's ambiguity across midnight.

Alternative representations: a rectangular one-dimensional array can use index = floor * slotsPerFloor + slot only when every floor has the same capacity. With uneven floors, either allocate F * maxCapacity cells (wasting cells and requiring validity checks), or build a prefix-offset array where index = offsets[floor] + slot. Known floor/slot lookup is O(1) with prefix offsets, using O(N+F) space and O(F) setup. Recovering a floor from a flat index is O(F) by linear offsets or O(log F) by binary search. Jagged arrays use O(N+F) space and O(1) known-coordinate access. ArrayList<String[]> also uses O(N+F) space, O(1) get(floor)[slot], and supports amortized O(1) appending of a row; each row still has fixed length. Every representation takes O(N) worst-case allocation by scanning. Strings store only occupant identity; the integrated Slot objects additionally carry suitability and controlled occupancy.

Task 3

Allocation uses nested loops in floor order and slot order. In the integrated model continue skips occupied or unsuitable slots and return stops both loops on the first successful allocation. No-suitable-slot exhaustion reports an error; it never creates an unallocated active ticket. The supplied exercise returns null on exhaustion. release() validates the floor before indexing its row, validates the slot against that row length, and returns false when already free. break is useful when only the inner loop should stop; return is clearer for returning the first match from nested loops.

The pseudocode uses INPUT, OUTPUT, BEGIN/END, IF/ENDIF and FOR/ENDFOR to state the algorithm and control flow.

```
ALGORITHM AllocateSuitableSlot
INPUT vehicle, needsCharging, slots
OUTPUT position or failure
BEGIN
  FOR floor FROM 0 TO length(slots) - 1 DO
    FOR slot FROM 0 TO length(slots[floor]) - 1 DO
      IF occupied OR NOT suitable(vehicle, needsCharging) THEN
        CONTINUE
      ENDIF
      store vehicle in slots[floor][slot]
      RETURN (floor, slot)
    ENDFOR
  ENDFOR
  RETURN failure
END
ALGORITHM ComputeFee
INPUT vehicle, entry, exit, pass, role, kWh
OUTPUT total fee
BEGIN
  IF exit < entry OR kWh < 0 THEN REJECT ENDIF
  minutes <- ceiling(duration(entry, exit) in minutes)
  hours <- minutes DIV 60
  IF minutes MOD 60 > 0 THEN hours <- hours + 1 ENDIF
  parking <- hours * vehicle.hourlyRate
  IF valid pass covers registration, role, entry and exit THEN
    parking <- 0
  ENDIF
  electricity <- 0
  IF charging vehicle AND valid charging booking THEN
    electricity <- 12 * kWh
  ENDIF
  RETURN parking + electricity
END
```

The selected hourly rule uses if for rounding and pass decisions; Main uses switch for input category selection. A different first-hour/additional-hour rule would use IF hours=0 THEN 0 ELSE firstRate + (hours-1)*extraRate. If a daily maximum C were specified, full-day blocks and the last partial day would each be capped at C; it is not silently introduced into this implementation.

Task03/Task03.java allocates the first free slot, validates release requests and matches all six expected output lines. The class name is Task03; the allocation and release behaviour follows the supplied program. The integrated facility adds suitability and duplicate checks.

Alternative: keep freeCount[f] and a stack of free indexes for each floor. allocate scans freeCount until a suitable nonempty floor, pops one index, marks occupied and decrements its count; release first validates that the slot is occupied, clears it, pushes once and increments. For F floors of S slots, the original exercise allocate is O(FS) worst-case and release O(1), with O(FS) storage and O(1) extra workspace. Per-floor stacks reduce allocate to O(F), release O(1), with O(FS+F) extra free-index/count storage. A global stack/set of nonempty floors reduces arbitrary allocation to O(1), but preserving first-floor order may need an ordered structure and separate suitability pools. A LIFO stack does not necessarily preserve the original first-free-slot order. Integrated duplicate detection and locating a free ticket-array cell remain O(N), even if slot selection is optimized.

Task03.java

```
public class Task03 {
   private final String[][] slots;
   public Task03(int[] slotsPerFloor) {
      slots = new String[slotsPerFloor.length][];
      for (int f = 0; f < slotsPerFloor.length; f++) {
         slots[f] = new String[slotsPerFloor[f]];
      }
   }
   public int[] allocate(String vehicleNo) {
      for (int f = 0; f < slots.length; f++) {
         for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
               slots[f][s] = vehicleNo;
               return new int[]{f, s};
            }
         }
      }
      return null;
   }
   public boolean release(int floor, int slot) {
      if (floor < 0 || floor >= slots.length || slot < 0 || slot >= slots[floor].length
            || slots[floor][slot] == null) return false;
      slots[floor][slot] = null;
      return true;
   }
   public static void main(String[] args) {
      Task03 p = new Task03(new int[]{2, 1});
      System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
      System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
         + p.release(5, 0));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
   }
}
```

Executed output:

[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]

Task 3

```
java -cp "build/Task03;build" Task03
```

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Process exit code: 0

Task 4

Vehicle has private final regNo, getRegNo(), abstract ratePerHour(), calculateFee(long), calculateFee(LocalDateTime,LocalDateTime), equals(), hashCode() and toString(). TwoWheeler and Car specialize rates; ElectricCar implements chargingFee(). Slot has final floor, number and charging, plus mutable private vehicle; it exposes isFree(), isSuitable(), validated occupy() and release(). ParkingTicket has final id, vehicle, slot, entry, role and pass with getters. ParkingFacility owns Slot[][], ParkingTicket[], PricingRule and collection; enter()/exit() enforce transactions. Pass checks coverage; PricingRule calculates parking fees. MonthlyPass owns registration and YearMonth.

ParkingTicket(Vehicle, Slot, LocalDateTime) chains with this(v,s,entry,"VISITOR",Pass.none()) to the full parameterized constructor. The short constructor supplies defaults without repeating validation. this.vehicle = v distinguishes the current object's field from an argument. Slot and Vehicle constructors reject invalid values so a usable object cannot be created with missing identity or an invalid position.

Java supplies a default no-argument constructor only when no constructor is declared. These domain classes deliberately have no no-argument constructors: an unidentified vehicle or positionless ticket would be invalid. A teaching-only default-plus-setters approach would temporarily violate those invariants. Constructor overloading provides alternative valid creation paths, not permission to skip required information. A duplicate ticket is a view/copy of the same issued ticket ID; a public copy constructor issuing a new ID would misleadingly represent a second admission. Therefore no copy constructor is provided. An immutable ticket reference may be reprinted without mutating admission state.

Conceptually local reference variables and call frames are on the thread stack and instances/arrays are on the heap; the JVM may optimize placement through escape analysis. A reference is not the object. Two references can refer to one Vehicle, as the ticket and occupied slot do; clearing only one reference does not free it.

Alternative: ParkingTicket t = new ParkingTicket(); t.setVehicle(v); t.setSlot(s); t.setEntry(in) requires a final validate step and exposes partial state unless setters are restricted behind a builder. A static factory ParkingTicket.issue(v,s,in) can call the validated constructor and give creation a descriptive name. Factories can cache or return subtypes, but ticket issue must always create a new identity. All three approaches create n fixed-size tickets in O(n) time and O(n) retained space, with O(1) per object. Setters add a constant number of calls/checks; a validating factory adds one delegating call, not a different Big-O. Registration length and UUID/string storage are bounded in this analysis.

Task 5

ParkingTicket has a static AtomicLong NEXT shared by every ticket and every facility in the JVM. static final RUN is initialized once to a UUID and acts as a run namespace. id combines RUN with NEXT.incrementAndGet(). A per-ticket counter starting at zero would repeat values. issuedCount() is static because it describes the class-wide sequence, not one admission. Static initialization expressions suffice; a static block would be appropriate for multi-step constant/configuration setup but is unnecessary here. Occupancy and collection belong to each facility; making them static would mix north and south gate totals. occupancy() counts active entries rather than maintaining an error-prone second mutable counter.

enter() returns a ParkingTicket reference. Passing Vehicle or ParkingTicket passes a copy of the reference value: Java is always pass-by-value. Mutating the referenced mutable object can be visible to the caller, but assigning a parameter to null cannot clear the caller's variable. Tickets and registrations are immutable; the facility owns the array of active references.

After successful exit, active[i] is cleared and the slot releases its Vehicle. The ticket becomes eligible for garbage collection only when no live caller, history list or other reachable reference retains it. In Task05 the local reference is explicitly cleared after exit. Eligibility does not guarantee immediate collection, and System.gc() would only request it. Vehicle collection likewise depends on every remaining reference. No finalizer is used.

Alternative: a counter field per ParkingFacility uses O(k) counter storage for k facilities, rather than O(1) for the shared static counter, independent of n tickets. Local numbers 1,2,... overlap between north and south. IDs such as NORTH-1 and SOUTH-1 with a persistent unique facility prefix solve this if each facility persists its next number. A central durable sequence solves global numbering across processes. Each ticket still stores one bounded ID, O(n) total, under either approach. The implemented UUID namespace makes restart collisions extremely unlikely but is not a durable database uniqueness constraint. The shared AtomicLong makes increment atomic; it does not make the whole facility thread-safe. Allocation, exit and count operations are O(N) in the array implementation; number generation is O(1).

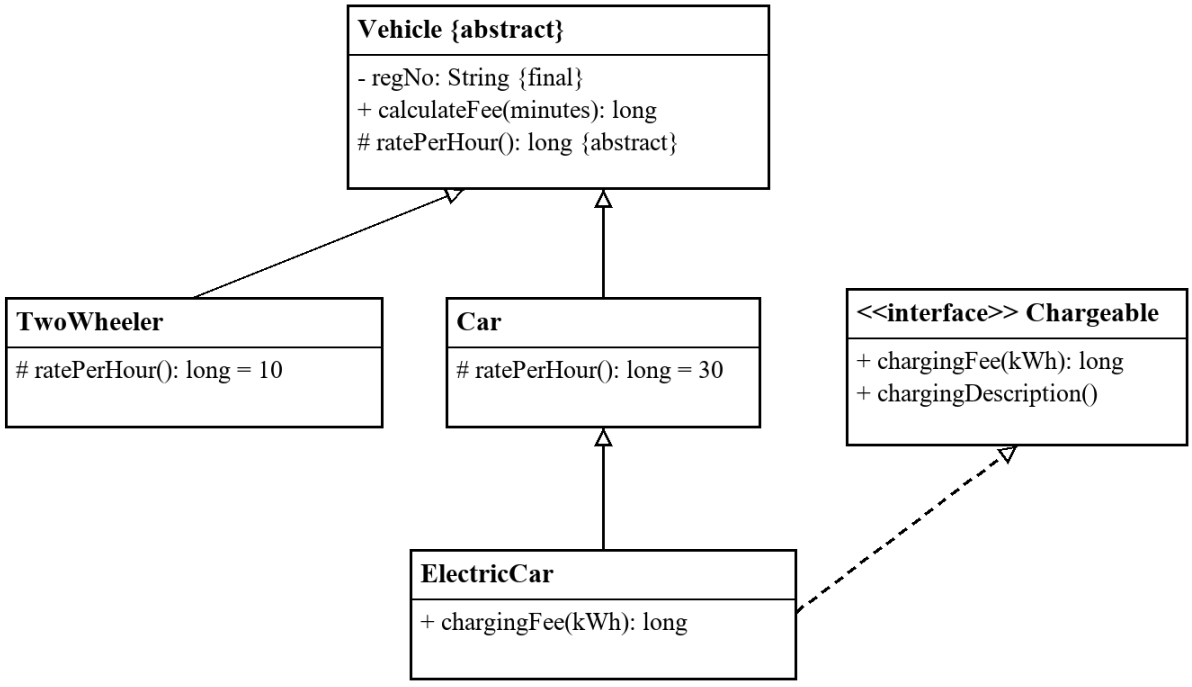
Task 6

Vehicle is the abstract superclass. TwoWheeler and Car extend Vehicle, forming hierarchical inheritance. ElectricCar extends Car, forming the multilevel path Vehicle -> Car -> ElectricCar. Java gives each class one direct superclass. Vehicle supplies registration, identity and fee overloads; TwoWheeler adds Rs 10/hour; Car adds Rs 30/hour; ElectricCar inherits the car rate and adds charging through Chargeable. The private registration field is initialized by Vehicle and accessed through getRegNo(), never directly by a subclass.

Each child constructor calls super(regNo). Constructor execution for new ElectricCar("TN01E1") runs Object, Vehicle, Car, ElectricCar in that order; argument evaluation occurs before the called constructor body. Task06's separate TraceVehicle/TraceCar/TraceElectricCar classes print 1 Vehicle constructor, 2 Car constructor, 3 ElectricCar constructor without inserting debugging output into the shared domain classes. super.method() can deliberately invoke the parent implementation when a subclass augments it; protected methods are available to subclasses but the registration remains private. Using a super field access would still require that field to be accessible, so exposing regNo merely to demonstrate it is inappropriate.

Users are represented by a validated role and a composed Pass instead of a Student/Staff/Visitor inheritance hierarchy. A user can change entitlement without changing class, and no other distinct user behaviour is needed.

Alternative: Vehicle has-a VehicleType(registration-independent name, hourlyRate). Example: Vehicle v = new Vehicle(reg, new VehicleType("CAR",30)); fee = ceilHours(minutes) * v.type.rate. Adding a simple new rate can then be a configuration entry without a new subclass. A subclass hierarchy adds a type class and factory registration; composition adds a policy/configuration object and factory registration. Both charge in O(1) for fixed rules; the inheritance vehicle stores O(1) state and dispatch metadata, while composition adds O(1) reference per vehicle plus O(k) shared type objects. A fresh type object per vehicle uses O(n) type objects, which can be avoided by sharing immutable types. Inheritance clearly demonstrates is-a relationships for this assignment; pricing composition is retained for changes that do not fit the type hierarchy.



Task 7

TwoWheeler.ratePerHour() and Car.ratePerHour() override the abstract Vehicle method. Vehicle.calculateFee() calls that method, so the correct rate is selected even though the calculation is inherited. ElectricCar implements Chargeable.chargingFee() at Rs 12/kWh. Different rates and optional charging are the reasons for these specializations; duplicating all the fee arithmetic in every subclass would spread rounding rules.

@Override asks the compiler to check that a superclass/interface method is actually overridden. Name and parameter types must match; primitive return types must match, and reference returns may be covariant. Access cannot be narrowed; checked exceptions cannot be widened. A static method is hidden, not dynamically overridden; private methods are not inherited overrides. The completed exercise's chargingFee satisfies the interface even without an annotation; shared source uses @Override.

regNo is final to keep vehicle identity stable in sets/maps. equals() and hashCode() are final in the shared base so a subclass cannot break symmetry by adding category to equality. ParkingTicket is a final immutable class; its fixed fields cannot be reassigned. The final slots/active references prevent replacing the arrays, but their elements remain mutable. These uses of final protect specific invariants and do not make the entire application immutable.

Vehicle.toString() returns runtime class name and registration. equals() accepts any Vehicle with the same normalized registration. hashCode() uses exactly that registration. Therefore a Car and ElectricCar with TN02B2 compare equal and collapse to one HashSet entry. This is appropriate because identity is registration, not a mutable category classification; duplicate active admission is prohibited. A jurisdiction with identical plates from different regions would require a combined region-and-registration identity.

Alternative: calculateFee(minutes) switches on a type enum and returns hours * rate selected by case. For k types, a linear if/else chain may take O(k) comparisons. A Java switch over a dense enum/int can be O(1), while string-switch implementation/collision behaviour is not an unconditional O(1) guarantee. Both use O(1) temporary space. Virtual dispatch is normally O(1) with respect to k. Adding a rate in a switch changes the central fee class (and enum/factory as applicable); adding a subclass leaves Vehicle fee arithmetic unchanged but adds a class and updates the external creation registry. In the inherited-template design the common fee formula remains in one place.

Task 8

Task08.java contains the completed fee program, with the supplied main executable statements preserved. Integer division and remainder calculate ceiling hours without floating arithmetic or minutes+59 overflow. For 130 minutes there are three billable hours: TwoWheeler 3*10=30.0, Car 3*30=90.0, ElectricCar 3*30 + 10*12=210.0. Every executed output line matches the four supplied expected lines, including true.

The last line is true because Vehicle.equals() checks instanceof Vehicle and the registration, not the concrete subclass. Both objects carry TN02B2, so they are one vehicle identity. The integrated implementation retains this rule and normalizes case; equal objects have equal hash codes. The supplied exercise is kept separate from the validating shared hierarchy.

Task08Overloading demonstrates calculateFee(long minutes) and calculateFee(LocalDateTime in, LocalDateTime out). The compiler resolves the overload from the argument types/arity: one long selects the first; two LocalDateTime arguments select the second. The date-aware overload avoids overnight ambiguity. Runtime dispatch then chooses overridden behaviour from the actual object, independent of the reference variable's declared type.

Trace: declared type Vehicle; actual object ElectricCar; call v.calculateFee(130). Vehicle.calculateFee executes because ElectricCar inherits that method. Its internal ratePerHour() call dispatches to Car.ratePerHour(), inherited by ElectricCar, returning 30. The fee is 90 before the Chargeable capability adds 120. Thus the dynamic dispatch point is specifically ratePerHour(), not a nonexistent ElectricCar.calculateFee override. A Vehicle[] can contain TwoWheeler, Car and ElectricCar through implicit upcasting; no unsafe cast is required to calculate parking fees.

Alternative: IF v instanceof ElectricCar THEN cast and compute EV fee ELSE IF v instanceof Car THEN car fee ELSE IF v instanceof TwoWheeler THEN bike fee. ElectricCar must be checked before Car because it is also a Car. With k category tests, a call takes O(k) worst-case and O(1) temporary space; virtual dispatch takes O(1) with respect to k. The type-check code grows O(k) and must be edited at each client site. Polymorphism moves specialization to the type and keeps the client loop unchanged. The single instanceof Chargeable test is a capability check rather than a chain enumerating every category.

Task08.java

```
interface Chargeable {
   double chargingFee(int kWh);
}
abstract class Vehicle {
   private final String regNo;
   protected Vehicle(String regNo) { this.regNo = regNo; }
   public String getRegNo() { return regNo; }
   protected abstract double ratePerHour();
   public double calculateFee(int minutes) {
      int hours = minutes / 60 + (minutes % 60 == 0 ? 0 : 1);
      return hours * ratePerHour();
   }
   @Override public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
   }
   @Override public int hashCode() { return regNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) { super(regNo); }
```

```
    @Override protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    public double chargingFee(int kWh) { return 12.0 * kWh; }
}
public class Task08 {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
            new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
                + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Executed output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Task 8

```
java -cp "build/Task08;build" Task08
```

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Process exit code: 0

Overloading output:

```
calculateFee(long): 90
calculateFee(LocalDateTime, LocalDateTime): 90
Declared Vehicle -> actual ElectricCar -> inherited calculateFee -> Car.ratePerHour
```

Task 9

Vehicle is abstract because there is no meaningful generic vehicle rate. Its constructor centralizes registration validation and its private field centralizes identity. ratePerHour() is abstract; concrete subclasses must supply it. Fee rounding, equality and registration access are reusable concrete methods. An abstract base may hold state and constructors while preventing direct instantiation.

Chargeable is an interface because charging is an optional capability, not the parent of every vehicle. ElectricCar can extend Car and implement Chargeable simultaneously. A class may implement further interfaces, such as an auditable or payable capability, without another superclass. These extra interfaces are illustrative and are not added as unused domain classes. Parking payment is a PricingRule/Pass decision; a Payable user interface is unnecessary when the operation already belongs to the facility.

Chargeable.chargingDescription() is a default method providing shared descriptive behaviour. Chargeable.validateUnits() is a static helper called using the interface name and rejects negative energy. Static interface methods are not inherited as instance methods. Pass.none() supplies a no-entitlement object rather than nullable checks. PricingRule is another interface; StandardPricing supplies the original fee policy. Interfaces specify a contract; instances supply behaviour.

Alternative: interface Vehicle could declare getRegNo(), ratePerHour() and a default calculateFee(). Each concrete implementation would then have to store and validate registration and implement equality/hashCode itself, or delegate those duties to another shared class. Interfaces have no per-instance fields or constructors and cannot enforce constructor-time state validation. If Chargeable became an abstract class holding charging state, ElectricCar could extend Chargeable or Car but not both, because Java forbids multiple class inheritance. Composition/delegation could recover both behaviours, adding an object/reference and plumbing. Default interface dispatch and abstract-class dispatch are both normally O(1) per call with respect to type count. Both can keep O(1) state per vehicle; using an extra delegate adds O(1) per-object space and possibly a shared O(k) policy pool. There is no universal exact byte comparison without inspecting the JVM object layout.

Task 10

parking.model contains Vehicle subclasses, Chargeable, Slot and ParkingTicket; parking.pass contains Pass and MonthlyPass; parking.pricing contains PricingRule and StandardPricing; parking.service contains ParkingFacility; parking.app contains Main. Packages match source directory names. javac --release 17 -d build followed by the shared source-file paths compiles the shared sources, and java -cp build parking.app.Main runs the app. Each task is compiled separately to avoid default-package class-name conflicts in the given exercises.

All model fields are private. Identity, configuration and ticket fields are final. Public getters expose scalar/immutable values; no public getter exposes the live active array or slots array. In the final revision, a ticket returns an immutable Position record rather than its mutable Slot; the facility alone resolves that position to its private slot. protected Vehicle constructor and ratePerHour() support subclass specialization without advertising rate mutation. Public enter()/exit() define facility operations. private find() is an implementation detail. The short example constructors in the supplied Task08 have package access as given; production constructors are public for access from parking.app. Demonstration trace classes are package-private because they are used only in Task06.

There is no arbitrary setRegNo()/setEntry() because changing identity or entry after admission would break billing and map/set semantics. Mutable state changes use validated domain setters: Slot.occupy(v) rejects null and occupied slots; release() rejects an already-free slot through false. The constructor rejects negative floor/number; isValidSlot() checks actual floor bounds. Vehicle rejects negative duration; charging validates kWh; the factory rejects unknown types. A validating setter is appropriate only when the field is legitimately mutable; immutability is stronger for identity. Invalid exit operations are checked before release/collection updates.

Task10 demonstrates Integer autoboxing/value comparison. Collections such as Map<Integer,...> would box int indexes; arrays use primitive int indexes directly. Integer == compares identity, so cached small values can misleadingly appear equal while larger values do not. Use equals() or unbox non-null values. Unboxing null throws NullPointerException; wrappers also introduce allocation/reference overhead. Task11's benchmark uses String keys and ParkingTicket values without needless boxing.

Alternative: public slot.vehicle permits callers to overwrite an occupied slot and public entry permits a caller to rewrite billing time. Every caller must repeat identical checks. A scalar range/negative check is O(1) per update; registration validation is O(L), bounded here by 15 characters. With m mutation sites, public-state validation must be maintained in O(m) places, compared with one controlled mutator/constructor. Overall p scalar updates still cost O(p) either way, but public fields cannot guarantee every call site validates. Public exposure of an array similarly bypasses business rules even if the array reference is final.

Task 11

The integrated implementation is in src/parking. Task11/Task11.java executes normal operation, rounding boundaries, rejected operations, passes, equality and ticket identity. Every assertion prints expected and actual values, and any mismatch throws AssertionError. Tests use explicit timestamps rather than the wall clock. The test table below records the expected and actual result of each executed assertion.

Hand calculation: a two-wheeler, car and EV each park for 130 minutes. Three started hours give Rs 30 + Rs 90 + Rs 90; EV electricity adds 10*12 = Rs 120. Total = Rs 330. Tests assert each exit, total collection 330 and zero remaining occupancy. Exact-hour 60 minutes is Rs 30, 61 minutes is Rs 60, zero is Rs 0. A positive second or nanosecond still counts as a started hour. Normal, full-capacity, duplicate registration, invalid floor/slot, expired pass, visitor monthly misuse, unknown type and failed-exit atomicity cases are checked. The standalone Task03 execution additionally proves release/re-release and reallocation behaviour.

Encapsulation is supported by private arrays/fields and validated operations. Inheritance supplies shared registration/fee behaviour. Polymorphism chooses rates without client category chains; Chargeable is a capability. Abstraction separates Pass and PricingRule from the facility. A limitation is in-memory, single-threaded state: tickets and collection are not persisted and admission is not synchronized. Different independent facilities do not enforce campus-wide active-registration exclusivity. The UUID prefix is probabilistic across process restarts; strict permanent ticket uniqueness requires durable state. These are explicit limits rather than claims of production readiness.

Allocation and exit scan a fixed-capacity array. Let N be total capacity and n active vehicles: lookup and insertion are O(N) worst-case even when n is small, because empty entries remain. Full dense occupancy gives N=n. release of a known position is O(1). Array storage reserves O(N) references; retained tickets/vehicles use O(n). Slot storage is O(N+F).

Alternative exit with ArrayList: FOR ticket IN activeList; IF ticket.reg=key THEN calculate, release slot and remove ticket; ENDIF. Lookup is O(n), removal shifts O(n), total O(n); storage O(n) with spare capacity. HashMap<String,ParkingTicket>: t=active.get(normalizedReg); validate and calculate before mutation; release t.slot; active.remove(key). Lookup/removal average O(1) with bounded registration length and a healthy hash distribution, worst-case O(n) under general hashing assumptions (modern Java may treeify some collision bins). Hashing strings costs O(L) initially. HashMap uses O(n) space with buckets/nodes and higher constant overhead than array references. Replacing the map does not improve a separate O(N) slot scan.

Task11/Task11Benchmark.java measures last-element successful lookup at n=20,150,500, 10000 warm-up and 20000 timed repetitions per structure. Each lookup result reaches a volatile sink. All structures share the same prebuilt tickets, and setup is outside timing. It measures lookup alone, not fee computation/removal or memory bytes. Results vary with JIT, CPU and run order and do not establish formal Big-O. Exact retained bytes were not measured; the space comparison is structural. Measured timings are listed below.

GitHub repository: https://github.com/192521143simats-sketch/CSA09-192521143-Assignment1

Executed tests: 43 passed, 0 failed.

Test	Expected	Actual	Result
zero minutes	0	0	PASS
exact hour	30	30	PASS
partial hour	60	60	PASS
130 minutes	90	90	PASS
positive second	30	30	PASS
fractional second	30	30	PASS
two wheeler	30	30	PASS
EV charging	120	120	PASS
negative duration	rejected	rejected	PASS
negative kWh	rejected	rejected	PASS
unknown type	rejected	rejected	PASS
invalid registration	rejected	rejected	PASS
first slot	0	0	PASS
duplicate registration	rejected	rejected	PASS
last general slot occupied	rejected	rejected	PASS
full occupancy	3	3	PASS
invalid floor	rejected	rejected	PASS
invalid slot index	false	false	PASS
invalid floor index	false	false	PASS
exit before entry	rejected	rejected	PASS
invalid exit retains occupancy	3	3	PASS
non-EV charging	rejected	rejected	PASS
car exit	90	90	PASS
bike exit	30	30	PASS
EV exit	210	210	PASS
hand collection 90+30+210	330	330	PASS
released occupancy	0	0	PASS
unknown exit	rejected	rejected	PASS
release occupied	true	true	PASS
release already free	false	false	PASS
reallocation	0	0	PASS

Test	Expected	Actual	Result
ticket never reused	false	false	PASS
monthly pass free	0	0	PASS
expired pass pays	30	30	PASS
visitor cannot use monthly	30	30	PASS
pass does not waive charging	120	120	PASS
monthly expiry midnight	false	false	PASS
pass bound to registration	false	false	PASS
equal across subclass	true	true	PASS
equal hashes	-1816604853	-1816604853	PASS
zero capacity	rejected	rejected	PASS
unique between facilities	false	false	PASS
isolated occupancy	1	1	PASS

Ticket lookup measurements (nanoseconds per lookup):

Vehicles	Array	ArrayList	HashMap
20	112	288	63
150	462	514	22
500	2109	2570	19

Task 12

A. The changed requirements add flat-fee e-bikes, delivery vans with double car pricing only during 09:00-17:00, visitor one-day passes, and five unequal floors. Vehicle category, pricing, entitlement and facility configuration are affected; ticket identity, allocation, release and encapsulation rules remain the same. A rectangular index f*S+s assumes equal floor sizes. The existing jagged Slot[][] accepts five row lengths directly, so no allocation rewrite is needed.

B. The original hierarchy can add EBike and DeliveryVan subclasses without changing Vehicle, Car, TwoWheeler or ElectricCar. Simple hourly categories fit ratePerHour(); a time-dependent van rule does not fit a method receiving only minutes. The existing PricingRule interface already receives entry and exit times, so a new policy can specialize van pricing without moving time-window logic into ParkingFacility. Identity/rounding stay shared; the changed window rule is centralized in one new policy. Pass is already a capability contract, so OneDayPass is an additional implementation. The original Main factory remains a baseline app with its original three categories; a separate Task12 constructs the extended categories without altering its switch.

C. Decision: RETAIN the core model and MODIFY the application by extension and configuration. Add EBike extends Vehicle, DeliveryVan extends Car, ChallengePricing implements PricingRule, and OneDayPass implements Pass. Configure capacities {2,3,4,2,5}. Two existing classes are tightened: ParkingTicket.getSlot() now returns an immutable Position record, and ParkingFacility uses that position to release its privately held slot. This closes a mutable-reference leak while retaining admission and pricing behaviour. Existing model/service classes changed: 2. New shared classes: 4; the new entry point and tests provide execution evidence. If one combined interactive category factory were required, its registry would need two entries; this separate deterministic entry point avoids changing the baseline factory.

Assumptions for underspecified prices: e-bike flat parking fee is Rs 20 for any positive stay, zero for zero duration. One-day pass is registration-bound, visitor-only, and covers one calendar date; both endpoints must lie on that date, so exit exactly at next midnight is not covered. Purchase price is not specified and is not included in collection. Monthly/day passes waive parking only. Delivery vans use Rs 30 per off-peak hour and Rs 60 per peak hour; parking is partitioned by each calendar day's 09:00 and 17:00 boundaries, charging proportional elapsed minutes in each band and rounding the final rupee total upward. This deliberately avoids independently rounding every band to a full hour or doubling an entire mixed stay. It is a changed rule specifically for vans; other categories retain started-hour billing. Tests use minute-aligned van timestamps; arbitrary seconds are priced proportionally too.

Window algorithm: cursor=entry; WHILE cursor<exit: select next boundary among 09:00,17:00,next midnight; end=min(boundary,exit); rate=60 if cursor in [09:00,17:00), otherwise 30; add elapsedSeconds*rate; cursor=end; ENDWHILE; return ceiling(total weighted seconds /3600), including subsecond remainder. This runs O(D) time over D calendar days and O(1) workspace. The standard fee remains O(1), while jagged allocation is O(N) and ticket lookup O(N). Flat e-bike billing and pass checks take O(1).

D. Evidence: existing changed classes are ParkingTicket and ParkingFacility (2); four new shared classes implement the additional categories, pass and pricing. Original Task11 is rerun after extension, and Task12Tests checks e-bikes, peak/off-peak vans, mixed-window and overnight intervals, zero/negative durations, day-pass validity/expiry, five unequal floors, full capacity and charging suitability. Hand checks: 08:00-10:00 van = 30+60=90; 16:30-17:30 = 30+15=45; 09:00-17:00 = 8*60=480; 17:00-09:00 next day = 16*30=480. A further plain hourly type needs one subclass and creation at the app boundary; a new time-based policy needs a new PricingRule and explicit policy selection. Trade-offs are the longer O(D) van calculation, more small policy classes

and a deliberate documented rounding difference. These are preferable to rewriting the allocation algorithm or spreading time windows over all vehicle classes. The final recommendation follows the two small encapsulation changes, four extension classes and executed regression/new-category results.

Executed tests: 31 passed, 0 failed.

Test	Expected	Actual	Result
van mixed morning	90	90	PASS
van mixed evening	45	45	PASS
van peak day	480	480	PASS
van off-peak night	480	480	PASS
van exact 09 boundary	60	60	PASS
van exact 17 boundary	30	30	PASS
van zero duration	0	0	PASS
van fractional minute	1	1	PASS
van subsecond	1	1	PASS
van whole 24 hours	960	960	PASS
van two days	1920	1920	PASS
van negative interval	rejected	rejected	PASS
e-bike zero	0	0	PASS
e-bike positive minute	20	20	PASS
e-bike long flat stay	20	20	PASS
e-bike date overload	20	20	PASS
e-bike negative	rejected	rejected	PASS
day pass valid	true	true	PASS
day pass midnight expiry	false	false	PASS
day pass wrong role	false	false	PASS
day pass wrong plate	false	false	PASS
five unequal floors total	16	16	PASS
floor 4 last valid slot	true	true	PASS
floor 0 invalid slot	false	false	PASS
all general slots full	rejected	rejected	PASS
all 16 slots occupied	16	16	PASS
all charging slots full	rejected	rejected	PASS
day pass exit free	0	0	PASS

Test	Expected	Actual	Result
day pass expired pays	480	480	PASS
day pass electricity retained	120	120	PASS
facility uses van policy	90	90	PASS

Task12.java output:

Five floor capacities: [2, 3, 4, 2, 5]; total=16
E-bike flat fee for 2 hours: Rs 20
Van 08:00-10:00 (30 + 60): Rs 90
Visitor one-day pass parking: Rs 0
Total collection: Rs 110
Occupancy after exits: 0

Task 12

```
java -cp "build/Task12;build" Task12
```

```
Five floor capacities: [2, 3, 4, 2, 5]; total=16
E-bike flat fee for 2 hours: Rs 20
Van 08:00-10:00 (30 + 60): Rs 90
Visitor one-day pass parking: Rs 0
Total collection: Rs 110
Occupancy after exits: 0
```

Process exit code: 0